

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 5
по дисциплине: Вычислительная математика
по теме: Численное дифференцирование

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Четвертухин В. Р.

Белгород 2024

Цель работы: Изучить основные численные формулы дифференцирования, особенности их алгоритмизации.

Цель работы обуславливает постановку и решение следующих задач:

- 1) Рассмотреть теоретические основы численного дифференцирования для аппроксимации разных порядков.
- 2) Научиться выбирать формулы дифференцирования и алгоритмизировать их в зависимости от численной ситуации с вниманием к проблемам точности, численной стабильности и релевантности поставленной задачи.
- 3) Выполнить индивидуальное задание, закрепляющее на практике полученные знания и практические навыки (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления). Алгоритм выполнения индивидуального задания находится в разделе «Ход выполнения практической части лабораторной работы».
- 4) Отразить в отчете все полученные результаты, включая графики (при необходимости), тексты программ. Сделать выводы.

Задание 1) – 2) Рассмотреть теоретические основы численного дифференцирования для аппроксимации разных порядков.

Научиться выбирать формулы дифференцирования и алгоритмизировать их в зависимости от численной ситуации с вниманием к проблемам точности, численной стабильности и релевантности поставленной задачи.

1) Рассмотреть теоретические основы численного дифференцирования для аппроксимации разных порядков.

▼ Прямая разность

```
[6] %writefile lab5_pz.rs
fn forward_difference(f: fn(f64) -> f64, x: f64, h: f64) -> f64 {
    (f(x + h) - f(x)) / h
}

fn main() {
    fn func(x: f64) -> f64 {
        x.powi(2)
    }

    let x_value = 2.0;
    let step_size = 0.001;
    let result = forward_difference(func, x_value, step_size);
    println!("Forward difference approximation of f'(2): {}", result);
}
```

Writing lab5_newton.rs

```
[7] !rustc lab5_pz.rs
```

```
[8] !./lab5_pz
```

```
Forward difference approximation of f'(2): 4.000999999999699
```

- Обратная разность

```
[11] %\writefile lab5_oz.rs
fn backward_difference(f: fn(f64) -> f64, x: f64, h: f64) -> f64 {
    (f(x) - f(x - h)) / h
}

fn main() {
    fn func(x: f64) -> f64 {
        x.powi(2)
    }

    let x_value = 2.0;
    let step_size = 0.001;
    let result = backward_difference(func, x_value, step_size);
    println!("Backward difference approximation of f'(2): {}", result);
}
```

Overwriting lab5_oz.rs

▼ Центральная разность

```
✓ 0 CSC [14] %%writefile lab5_cz.rs
fn central_difference(f: fn(f64) -> f64, x: f64, h: f64) -> f64 {
    (f(x + h) - f(x - h)) / (2.0 * h)
}

fn main() {
    fn func(x: f64) -> f64 {
        x.powi(2)
    }

    let x_value = 2.0;
    let step_size = 0.001;
    let result = central_difference(func, x_value, step_size);
    println!("Central difference approximation of f'(2): {}", result);
}
```

Writing lab5_cz.rs

```
✓ 0 CSC [15] !rustc lab5_cz.rs
```

```
✓ 0 CSC [16] !./lab5_cz
```

Central difference approximation of f'(2): 3.9999999999995595

▼ Центральная разность второй производной

```
✓ 0 CSC [17] %%writefile lab5_cz2.rs
fn second_derivative_central_difference(f: fn(f64) -> f64, x: f64, h: f64) -> f64 {
    (f(x + h) - 2.0 * f(x) + f(x - h)) / (h * h)
}

fn main() {
    fn func(x: f64) -> f64 {
        x.powi(2)
    }

    let x_value = 2.0;
    let step_size = 0.001;
    let result = second_derivative_central_difference(func, x_value, step_size);
    println!("Central difference approximation of f''(2): {}", result);
}
```

Writing lab5_cz2.rs

```
✓ 0 CSC [18] !rustc lab5_cz2.rs
```

```
✓ 0 CSC [19] !./lab5_cz2
```

2) Выполнить индивидуальное задание, закрепляющее на практике полученные знания и практические навыки (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления). Алгоритм выполнения индивидуального задания находится в разделе «Ход выполнения практической части лабораторной работы».

Алгоритм на языке Rust для вычисления первой и второй производных функции $f(x) = x^3 \cos(0.5x) - \cosh(0.5x)$ с использованием центральной разности:

```
%writefile lab5_my_exercise.rs

fn f(x: f64) -> f64 {
    x.powi(3) * (0.5 * x).cos() * (0.5 * x).cosh()
}

fn central_difference(f: fn(f64) -> f64, x: f64, h: f64) -> f64 {
    (f(x + h) - f(x - h)) / (2.0 * h)
}

fn second_derivative_central_difference(f: fn(f64) -> f64, x: f64, h: f64) -> f64 {
    (f(x + h) - 2.0 * f(x) + f(x - h)) / (h * h)
}

fn main() {
    let x_value = 2.0;
    let step_size = 0.001;

    let result_first_derivative = central_difference(f, x_value, step_size);
    println!("Central difference approximation of f'(2): {}", result_first_derivative);

    let result_second_derivative = second_derivative_central_difference(f, x_value, step_size);
    println!("Central difference approximation of f''(2): {}", result_second_derivative);
}
```

Overwriting lab5_my_exercise.rs

```
[26] !rustc lab5_my_exercise.rs
```

```
[27] !./lab5_my_exercise
```

Central difference approximation of f'(2): 7.3507808671293695
Central difference approximation of f''(2): -1.9127602230994967

3) Выполнить индивидуальное задание, закрепляющее на практике полученные знания и практические навыки

```
%%writefile lab5_my_exercise2.rs
use std::f64::consts::{PI, E};

fn main() {
    let x: f64 = 1.5;

    let f = |x: f64| x.powi(3) * (0.5 * x).cos() * (0.5 * x).cosh();

    let exact_derivative = 3.0 * x.powi(2) * (0.5 * x).cos() * (0.5 * x).cosh()
        - 0.5 * x.powi(3) * (0.5 * x).sin() * (0.5 * x).cosh()
        + 0.5 * x.powi(3) * (0.5 * x).cos() * (0.5 * x).sinh();

    println!("Точное значение первой производной в точке {}: {}", x, exact_derivative);

    let first_order_diff = |f: &dyn Fn(f64) -> f64, x: f64, h: f64| {
        (f(x + h) - f(x)) / h
    };

    let second_order_diff = |f: &dyn Fn(f64) -> f64, x: f64, h: f64| {
        (f(x + h) - 2.0 * f(x) + f(x - h)) / h.powi(2)
    };

    let fourth_order_diff = |f: &dyn Fn(f64) -> f64, x: f64, h: f64| {
        (-f(x + 2.0 * h) + 8.0 * f(x + h) - 8.0 * f(x - h) + f(x - 2.0 * h)) / (12.0 * h)
    };

    let hs = [0.1, 0.01, 0.001];

    for &h in &hs {
        let first_order_approximation = first_order_diff(&f, x, h);
        let second_order_approximation = second_order_diff(&f, x, h);
        let fourth_order_approximation = fourth_order_diff(&f, x, h);

        // Абсолютные и относительные погрешности для каждого порядка аппроксимации
        let first_order_absolute_error = (first_order_approximation - exact_derivative).abs();
        let first_order_relative_error = first_order_absolute_error / exact_derivative.abs();

        let second_order_absolute_error = (second_order_approximation - exact_derivative).abs();
        let second_order_relative_error = second_order_absolute_error / exact_derivative.abs();
    }
}
```

```

        let fourth_order_absolute_error = (fourth_order_approximation -
exact_derivative).abs();
        let fourth_order_relative_error = fourth_order_absolute_error /
exact_derivative.abs();

        // Теоретическая оценка абсолютной погрешности
        let r = 0.5 * (f64::EPSILON.abs() / h.abs()) * (3.0 * x.powi(2) +
h.powi(2));

        println!("При h = {}: ", h);
        println!("Численное дифференцирование первого порядка:
приближенное значение = {}, абсолютная погрешность = {}, относительная
погрешность = {}, теоретическая оценка абсолютной погрешности = {}",
                first_order_approximation, first_order_absolute_error,
first_order_relative_error, r);

        println!("Численное дифференцирование второго порядка:
приближенное значение = {}, абсолютная погрешность = {}, относительная
погрешность = {}",
                second_order_approximation, second_order_absolute_error,
second_order_relative_error);

        println!("Численное дифференцирование четвертого порядка:
приближенное значение = {}, абсолютная погрешность = {}, относительная
погрешность = {}",
                fourth_order_approximation, fourth_order_absolute_error,
fourth_order_relative_error);

        println!();
    }
}

```

```

%%writefile lab5_my_exercise3.rs
use std::f64::consts::PI;
fn main() {
    let x: f64 = 1.5;

    let f = |x: f64| x.powi(3) * (0.5 * x).cos() * (0.5 * x).cosh();

    let exact_second_derivative = 6.0 * x * (0.5 * x).cos() * (0.5 *
x).cosh()
        - 1.5 * x.powi(2) * (0.5 * x).sin() * (0.5 * x).cosh()
        - 1.5 * x.powi(2) * (0.5 * x).cos() * (0.5 * x).sinh()
        + 0.25 * x.powi(3) * (0.5 * x).cos() * (0.5 * x).cosh();

    println!("Точное значение второй производной в точке {}: {}", x,
exact_second_derivative);

    let second_order_diff = |f: &dyn Fn(f64) -> f64, x: f64, h: f64| {
        (f(x + h) - 2.0 * f(x) + f(x - h)) / h.powi(2)
    };

    let mut min_error = std::f64::INFINITY;
    let mut optimal_h = 0.0;

    for h in [0.1, 0.01, 0.001, 0.0001].iter() {
        let approximation = second_order_diff(&f, x, *h);
        let absolute_error = (approximation -
exact_second_derivative).abs();

        if absolute_error < min_error {
            min_error = absolute_error;
            optimal_h = *h;
        }

        println!("При h = {}: приближенное значение = {}, абсолютная
погрешность = {}",
            h, approximation, absolute_error);
    }

    println!("Оптимальное значение h: {}", optimal_h);

    let optimal_approximation = second_order_diff(&f, x, optimal_h);
    let optimal_absolute_error = (optimal_approximation -
exact_second_derivative).abs();
    let optimal_relative_error = optimal_absolute_error /
exact_second_derivative.abs();

    println!("При оптимальном h = {}: приближенное значение = {},
абсолютная погрешность = {}, относительная погрешность = {}",
        optimal_h, optimal_approximation, optimal_absolute_error,
optimal_relative_error);
}

```



```

%%writefile lab5_my_exercise4.rs
use std::f64::consts::PI;
fn main() {
    let x: f64 = 1.5;

    let f = |x: f64| x.powi(3) * (0.5 * x).cos() * (0.5 * x).cosh();

    let exact_third_derivative = 6.0 * (0.5 * x).cos() * (0.5 * x).cosh()
        - 3.0 * x * (0.5 * x).sin() * (0.5 * x).cosh()
        - 3.0 * x * (0.5 * x).cos() * (0.5 * x).sinh()
        + 0.75 * x.powi(3) * (0.5 * x).cos() * (0.5 * x).cosh();

    println!("Точное значение третьей производной в точке {}: {}", x,
exact_third_derivative);

    let third_order_diff = |f: &dyn Fn(f64) -> f64, x: f64, h: f64| {
        (-f(x + 2.0 * h) + 2.0 * f(x + h) - 2.0 * f(x - h) + f(x - 2.0 *
h)) / (2.0 * h).powi(3)
    };

    let mut min_error = std::f64::INFINITY;
    let mut optimal_h = 0.0;

    for h in [0.1, 0.01, 0.001, 0.0001].iter() {
        let approximation = third_order_diff(&f, x, *h);
        let absolute_error = (approximation -
exact_third_derivative).abs();

        if absolute_error < min_error {
            min_error = absolute_error;
            optimal_h = *h;
        }

        println!("При h = {}: приближенное значение = {}, абсолютная
погрешность = {}",
            h, approximation, absolute_error);
    }

    println!("Оптимальное значение h: {}", optimal_h);

    let optimal_approximation = third_order_diff(&f, x, optimal_h);
    let optimal_absolute_error = (optimal_approximation -
exact_third_derivative).abs();
    let optimal_relative_error = optimal_absolute_error /
exact_third_derivative.abs();

    println!("При оптимальном h = {}: приближенное значение = {},
абсолютная погрешность = {}, относительная погрешность = {}",
        optimal_h, optimal_approximation, optimal_absolute_error,
optimal_relative_error);
}

```

Результат работы

Задание 1)

```
Точное значение первой производной в точке 1.5: 5.928416782758684
При h = 0.1:
Численное дифференцирование первого порядка: приближенное значение = 6.184968421745951, абсолютная погрешность = 0.2745517188933671, относительная погрешность = 0.8463737153080878, теоретическая оценка абсолютной погрешности = 0.000000000000750510764646858
Численное дифференцирование второго порядка: приближенное значение = 5.62983261559455, абсолютная погрешность = 0.2605848071561354, относительная погрешность = 0.84481446415661542
Численное дифференцирование четвертого порядка: приближенное значение = 5.928618473567878, абсолютная погрешность = 0.000193778883943347, относительная погрешность = 0.0008327292516933895

При h = 0.01:
Численное дифференцирование первого порядка: приближенное значение = 5.948753801442331, абсолютная погрешность = 0.028336298683647865, относительная погрешность = 0.004786208844868428, теоретическая оценка абсолютной погрешности = 0.000000000000749411643852269
Численное дифференцирование второго порядка: приближенное значение = 5.684843785569488, абсолютная погрешность = 0.2363729971892754, относительная погрешность = 0.83992586886254618
Численное дифференцирование четвертого порядка: приближенное значение = 5.928416722128255, абсолютная погрешность = 0.000000013361571381229612, относительная погрешность = 0.0000000032783568066454

При h = 0.001:
Численное дифференцирование первого порядка: приближенное значение = 5.922588864128815, абсолютная погрешность = 0.002841383651179437, относительная погрешность = 0.0001799161897651716, теоретическая оценка абсолютной погрешности = 0.000000000007494806516442831
Численное дифференцирование второго порядка: приближенное значение = 5.684285689763862, абсолютная погрешность = 0.23613189299482193, относительная погрешность = 0.83988428154358291
Численное дифференцирование четвертого порядка: приближенное значение = 5.928416782768445, абсолютная погрешность = 0.0000000000017612578862653483, относительная погрешность = 0.0000000000029748882463786835
```

Задание 2)

```
Точное значение второй производной в точке 1.5: 4.315908230210265
При h = 0.1: приближенное значение = 5.65983261559455, абсолютная погрешность = 1.3439243853842848
При h = 0.01: приближенное значение = 5.684043705569488, абсолютная погрешность = 1.3681354753591428
При h = 0.001: приближенное значение = 5.684285689763862, абсолютная погрешность = 1.3683773795535963
При h = 0.0001: приближенное значение = 5.684287973650726, абсолютная погрешность = 1.3683797434404603
Оптимальное значение h: 0.1
При оптимальном h = 0.1: приближенное значение = 5.65983261559455, абсолютная погрешность = 1.3439243853842848, относительная погрешность = 0.31138854528396925
```

Задание 3)

```
Точное значение третьей производной в точке 1.5: 1.402860364043537
При h = 0.1: приближенное значение = 1.2950523902633957, абсолютная погрешность = 0.1078079737801414
При h = 0.01: приближенное значение = 1.259088086624782, абсолютная погрешность = 0.1437722774187551
При h = 0.001: приближенное значение = 1.2587286213339155, абсолютная погрешность = 0.1441317427096216
При h = 0.0001: приближенное значение = 1.25865984301754, абсолютная погрешность = 0.14420052102599712
Оптимальное значение h: 0.1
При оптимальном h = 0.1: приближенное значение = 1.2950523902633957, абсолютная погрешность = 0.1078079737801414, относительная погрешность = 0.07684868468975835
```

Ссылка на блокнот:

<https://colab.research.google.com/drive/1onDodvyMObtTK5uSHGKfVoHGAh4KOV0?usp=sharing>

Вывод

Изучение численных формул дифференцирования и их алгоритмизация являются важной частью вычислительной науки и инженерии. Они позволяют эффективно решать задачи, связанные с производными функций, и находят применение в различных областях, таких как моделирование, оптимизация, анализ данных и машинное обучение.